

Experiment No.3

Implementation of Singly Linked List and Its Operations

Objective

The objective of this lab is to help students understand the concept of dynamic linear data structures and implement singly linked lists using C++.

After completing this experiment, students will be able to:

- understand the structure of a linked list node
- dynamically create nodes using pointers
- create a singly linked list
- traverse a linked list
- insert nodes at different positions
- search elements in a linked list
- delete nodes from a linked list

Theory

A linked list is a dynamic data structure in which elements are connected using pointers. Unlike arrays, elements in a linked list are not stored in contiguous memory locations.

Each element of a linked list is called a node. A node in a singly linked list contains two parts:

1. Data field – stores the actual value
2. Pointer field – stores the address of the next node

Example Linked List:

10 → 20 → 30 → 40 → NULL

Here the last node points to NULL which indicates the end of the list.

The pointer that stores the address of the first node is called the Head pointer.

Node Definition in C++:

```
class Node
{
public:
    int data;
```

```
Node* next;  
};
```

Algorithms:

1. Insert Node at Beginning

Step 1: Create a new node
Step 2: Store value in the node
Step 3: Set newNode->next = head
Step 4: Update head = newNode

2. Insert Node at End

Step 1: Create a new node
Step 2: If list is empty, make new node the head
Step 3: Otherwise traverse the list
Step 4: Attach the new node to the last node

3. Traversal

Step 1: Set pointer current = head
Step 2: While current $\neq$ NULL
Step 3: Print current data
Step 4: Move current to next node

4. Delete First Node

Step 1: Check if list is empty
Step 2: Store head in temporary pointer
Step 3: Move head to next node
Step 4: Delete temporary node

Example:

```
#include <iostream>  
using namespace std;  
  
class Node  
{  
public:  
    int data;  
    Node* next;  
};
```

```
void insertAtEnd(Node*& head, int value)
{
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;
    if(head == NULL)
    {
        head = newNode;
        return;
    }
    Node* current = head;
    while(current->next != NULL)
    {
        current = current->next;
    }
    current->next = newNode;
}

void insertAtBeginning(Node*& head, int value)
{
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void display(Node* head)
{
    Node* current = head;

    while(current != NULL)
    {
        cout << current->data << " -> ";
        current = current->next;
    }
    cout << "NULL" << endl;
}

bool search(Node* head, int key)
{
    Node* current = head;
    while(current != NULL)
    {
        if(current->data == key)
            return true;
        current = current->next;
    }
}
```

```
        return false;
    }
    void deleteFirst(Node*& head)
    {
        if(head == NULL)
        {
            cout << "List is empty" << endl;
            return;
        }
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    int main()
    {
        Node* head = NULL;
        insertAtEnd(head,10);
        insertAtEnd(head,20);
        insertAtEnd(head,30);
        cout<<"Linked List:"<<endl;
        display(head);
        insertAtBeginning(head,5);

        cout<<"After inserting at beginning:"<<endl;
        display(head);
        if(search(head,20))
            cout<<"20 Found"<<endl;
        else
            cout<<"20 Not Found"<<endl;

        deleteFirst(head);
        cout<<"After deleting first node:"<<endl;
        display(head);

        return 0;
    }
```

Output:

Linked List:

10 -> 20 -> 30 -> NULL

After inserting at the beginning:

5 -> 10 -> 20 -> 30 -> NULL

20 Found

After deleting the first node:

10 -> 20 -> 30 -> NULL

Lab Tasks:

Task 1: Create a singly linked list containing 5 nodes.

Task 2: Write a function to count the number of nodes.

Task 3: Write a function to insert a node after a given value.

Task 4: Write a function to delete the last node.

Task 5 (Challenge): Create a menu-driven linked list program with options:

1 Insert at Beginning

2 Insert at End

3 Display List

4 Search Element

5 Delete First Node

6 Exit